Name : Tanmay Mahesh Ahuja **Semester :** V **Branch :** AI&DS

Roll No : 03 **Batch :** T11 **Date :** 08-09-2026

Experiment No. 07

Aim : To implement monitoring using Prometheus / Grafana / Nagios

LO Mapped : LO5

LO : Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.

Theory :

Continuous monitoring is a process to detect, report, respond all the attacks which occur in its infrastructure. Once the application is deployed into the server, the role of continuous monitoring comes in to play. The entire process is all about taking care of the company's infrastructure and respond appropriately.

Why We Need Nagios tool?

Here, are the important reasons to use Nagios monitoring tool:

- Detects all types of network or server issues
- Helps you to find the root cause of the problem which allows you to get the permanent solution to the problem
- Active monitoring of your entire infrastructure and business processes
- Allows you to monitor and troubleshoot server performance issues
- Helps you to plan for infrastructure upgrades before outdated systems create failures
- You can maintain the security and availability of the service
- Automatically fix problems in a panic situation
- Nagios is the most popular, open source, powerful monitoring system for any kind of infrastructure. It enables organizations to identify and resolve IT infrastructure problems before they affect critical business processes. Nagios has the capability of monitoring applications, services and entire IT infrastructure.
- NRPE is known as Nagios Remote Plugin Executor. It is designed to execute plugins on remote systems. On Windows hosts, the equivalent remote agent is NSClient++, which listens for Nagios requests and executes local checks (disk, CPU, memory, services, etc.).

- NSClient++ runs as a Windows service on the remote host and waits for requests from the Nagios server (usually over the NRPE protocol, which NSClient++ also supports). When the Nagios server needs to check a resource on that host, it sends a command; NSClient++ executes the check locally and returns the result to Nagios.

Note on running Nagios on Windows

Nagios Core is built for Linux and does not install natively on Windows. The standard practice for a Windows machine is to run the Nagios Core server inside a Docker container (using Docker Desktop with the WSL2 backend), and to monitor Windows hosts using NSClient++, which is the Windows counterpart of NRPE. All commands below are Windows PowerShell / Command Prompt commands.

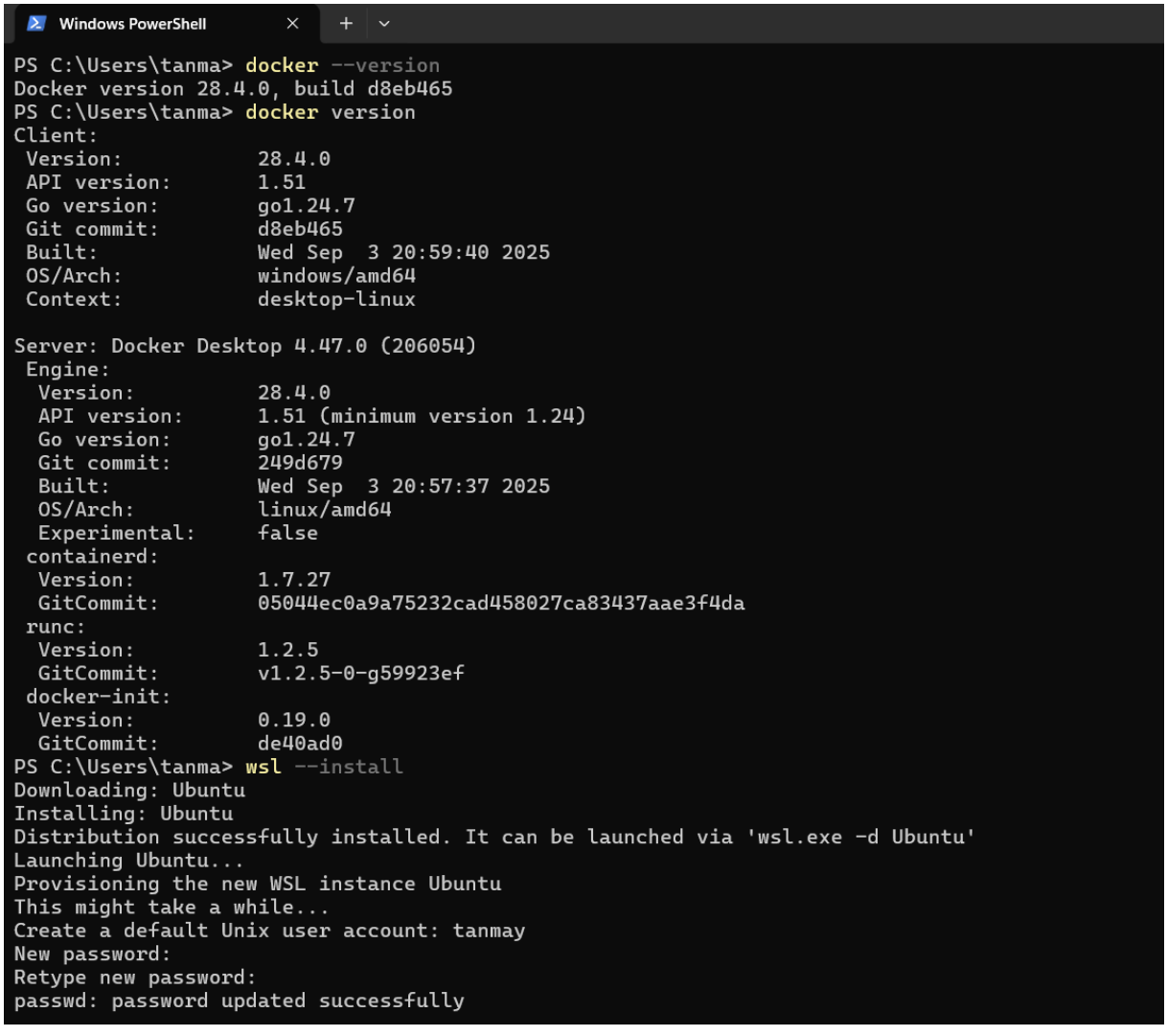
1 - Pre-requisite: Install Docker Desktop

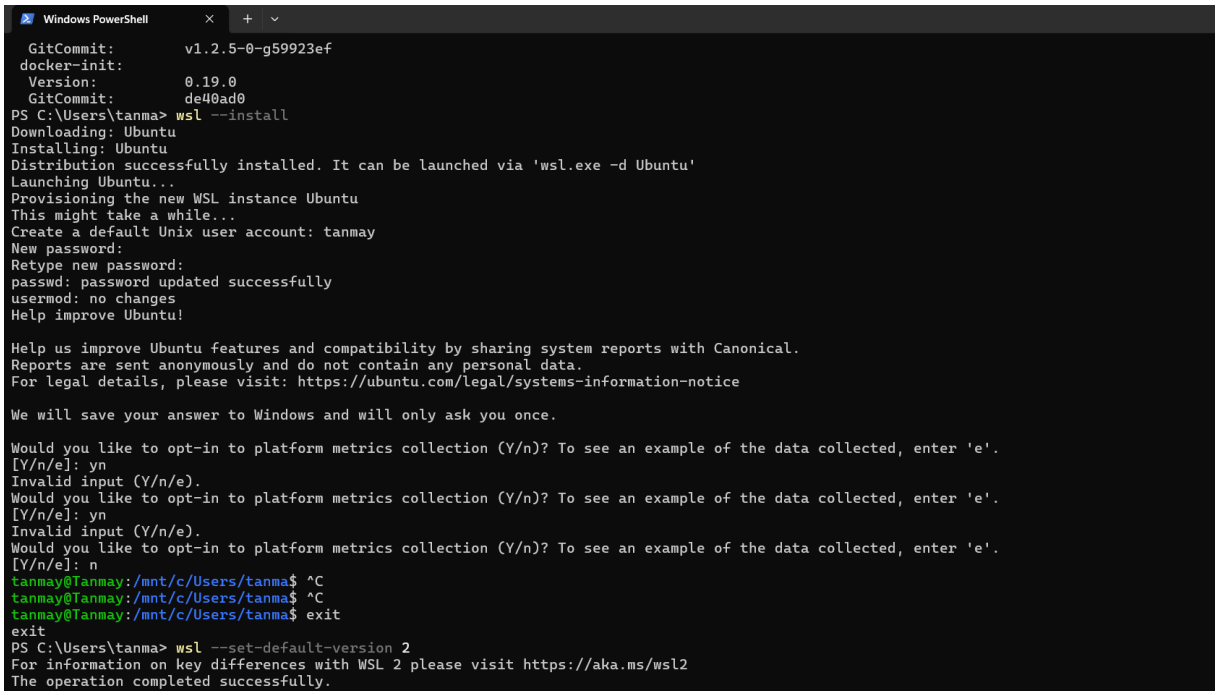
Download and install Docker Desktop for Windows and enable the WSL2 based engine. Verify the installation from PowerShell:

```
PS C:\Users\tanma> docker --version
PS C:\Users\tanma> docker version
```

Also make sure the WSL2 feature is enabled:

```
PS C:\Users\tanma> wsl --install
PS C:\Users\tanma> wsl --set-default-version 2
```





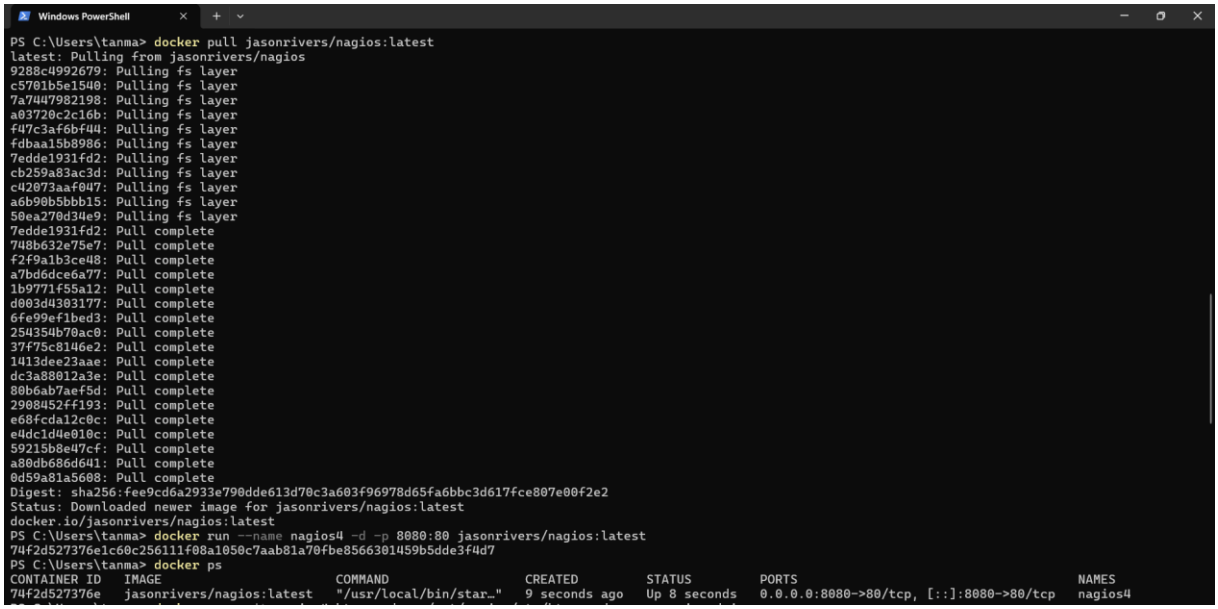
2 - Pull and Run the Nagios Core Docker Image

Pull a ready-made Nagios Core image and run it as a container, mapping the web port to the host.

```
PS C:\Users\tanma> docker pull jasonrivers/nagios:latest
PS C:\Users\tanma> docker run --name nagios4 -d -p 8080:80 jasonrivers/nagios:latest
```

Verify the container is up and running:

```
PS C:\Users\tanma> docker ps
```

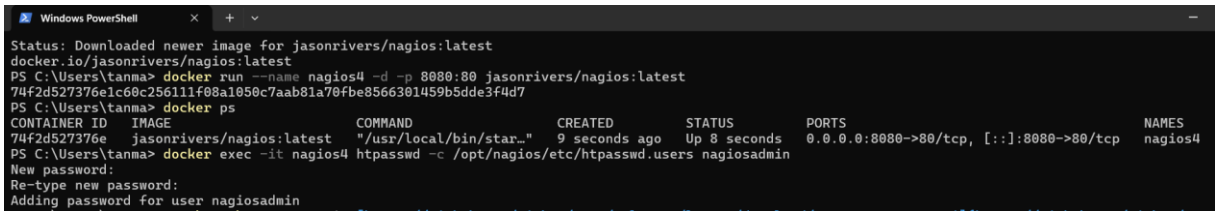


3 - Set the Apache/Nagios Admin Password

Set (or reset) the password for the nagiosadmin user inside the running container:

```
PS C:\Users\tanma> docker exec -it nagios4 htpasswd -c /opt/nagios/etc/htpasswd.users nagiosadmin
```

Enter and confirm the password when prompted (e.g. 123456).



4 - Access Nagios Web Interface

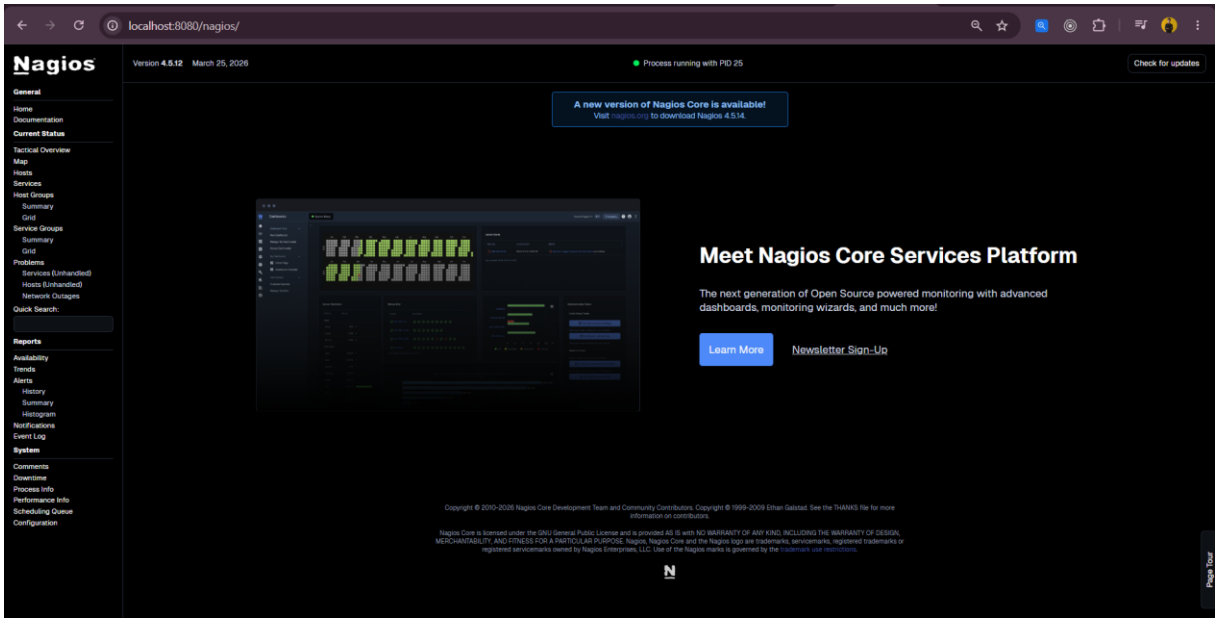
Access the Nagios web interface from a browser on the Windows host:

```
http://localhost:8080/nagios/
```

Prompting for Authentication Password -

username: nagiosadmin

Password: 123456 (which you entered while configuration)



5 - Install NSClient++ on the Windows Remote Host

On the Windows machine you want to monitor, download and silently install NSClient++ (the Windows equivalent of the NRPE agent):

```
PS C:\Users\tanma> Invoke-WebRequest -Uri "https://github.com/mickem/nscp/releases/latest/download/NSCP-0.5.2-x64.msi" -OutFile "NSCP.msi"
PS C:\Users\tanma> Start-Process msiexec.exe -ArgumentList '/i NSCP.msi ADDLOCAL=ALL MODULES=CheckExternalScripts,NRPEServer /qn' -Wait
```



6 - Configure NSClient++

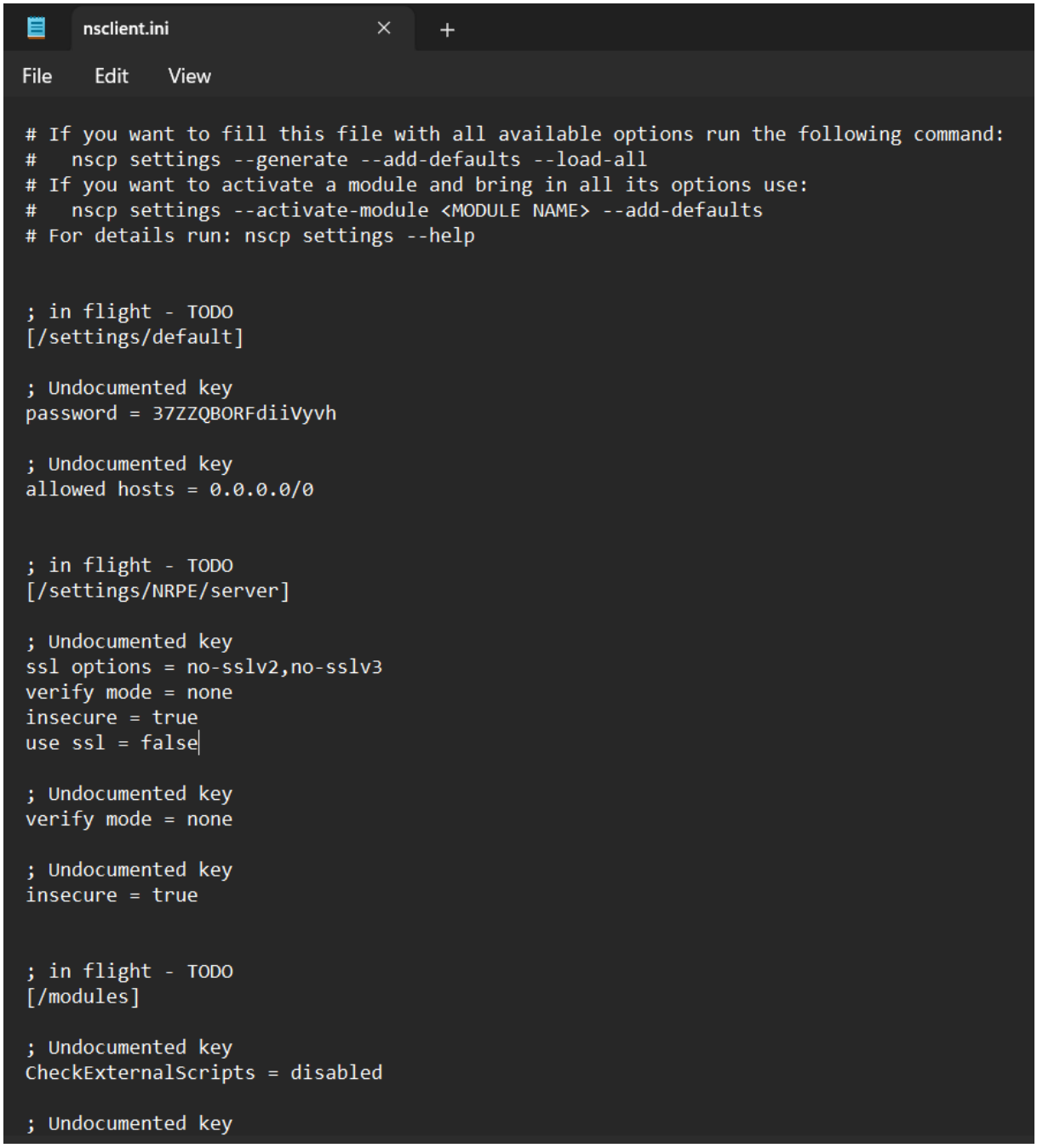
Edit the NSClient++ configuration file and add the Nagios server's IP address to the allowed hosts list:

```
PS C:\Users\tanma> notepad "C:\Program Files\NSClient++\nsclient.ini"
allowed hosts=127.0.0.1,192.168.0.107
```

Where 192.168.1.100 is your Nagios server (Docker host) IP address.

After saving the file, restart the NSClient++ service so the change takes effect:

```
PS C:\Users\tanma> Restart-Service nscp
```



```
; Undocumented key
CheckHelpers = disabled

; Undocumented key
CheckEventLog = disabled

; Undocumented key
CheckNSCP = disabled

; Undocumented key
CheckDisk = disabled

; Undocumented key
CheckSystem = disabled

; Undocumented key
NRPEServer = enabled
```

7 - Verify Connectivity from Nagios

From the Nagios server (inside the Docker container), run the NRPE check against the Windows remote host to confirm connectivity. Here 192.168.1.50 is the remote Windows host's IP address.

```
PS C:\Users\tanma> docker exec -it nagios4 /opt/nagios/libexec/check_nrpe -H 192.168.0.107 -n -2
I (0.5.2.35 2018-01-28) seem to be doing fine...
```

```
PS C:\Users\tanma> notepad "C:\Program Files\NSClient++\nsclient.ini"
PS C:\Users\tanma> Restart-Service nscp
PS C:\Users\tanma> docker exec -it nagios4 /opt/nagios/libexec/check_nrpe -H 192.168.0.107 -n -2
I (0.5.2.35 2018-01-28) seem to be doing fine...
```

8 - Add the Windows Host in Nagios

Create a configuration file for the new host inside the Nagios container. We also define a service that continuously checks whether the host is reachable (PING check).

```
PS C:\Users\tanma> $config = @"
define hostgroup {
    hostgroup_name  windows-servers
    alias           Windows Servers
}
define host {
    use             windows-server
    host_name       Windows_Host_001
    alias           Windows Host 001
    address         192.168.0.107
    register        1
```

```
}
define service{
   host_name               Windows_Host_001
   service_description     PING
   check_command           check_ping!100.0,20%!500.0,60%
   max_check_attempts      2
   check_interval          2
   retry_interval          2
   check_period            24x7
   check_freshness         1
   contact_groups          admins
   notification_interval   2
   notification_period     24x7
   notifications_enabled   1
   register                1
}
"@
PS C:\Users\tanma> $config | Out-File -FilePath "MyWindowsHost001.cfg" -Encoding
ASCII
PS C:\Users\tanma> docker exec -it nagios4 mkdir -p /opt/nagios/etc/servers/
PS C:\Users\tanma> docker cp MyWindowsHost001.cfg
nagios4:/opt/nagios/etc/servers/MyWindowsHost001.cfg
PS C:\Users\tanma> docker exec -it nagios4 sh -c "echo 'cfg_dir=/opt/nagios/etc/servers' >>
/opt/nagios/etc/nagios.cfg"
```

Verify the configuration file for errors, then restart the Nagios container so the changes take effect:

```
PS C:\Users\tanma> docker exec -it nagios4 /opt/nagios/bin/nagios -v
/opt/nagios/etc/nagios.cfg
PS C:\Users\tanma> docker restart nagios4
```

```
PS C:\Users\tanma> $config = @"-it nagios4 sh -c "echo 'cfg_dir=/opt/nagios/etc/servers' >> /opt/nagios/etc/nagios.cfg"
>> define hostgroup {cker exec -it nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg
>>     hostgroup_name  windows-servers
>>     alias           Windows Servers
>> }right (c) 2009-present Nagios Core Development Team and Community Contributors
>> define host {99-2009 Ethan Galstad
>>     use             windows-server
>>     host_name       Windows_Host_001
>>     alias           Windows Host 001
>>     address         192.168.0.107
>>     register        1a...
>> }ead main config file okay...
>> define service{ind any hostgroup matching 'windows-servers' (config file '/opt/nagios/etc/servers/MyWindowsHost001.cfg', starti
>>     host_name               Windows_Host_001
>>     service_description     PING names for host 'Windows_Host_001' (config file '/opt/nagios/etc/servers/MyWindowsHost001.cfg',
>>     check_command           check_ping!100.0,20%!500.0,60%
>>     max_check_attempts      2ig files!
>>     check_interval          2
>>     retry_interval          2
>>     check_period            24x7untered while processing the config files...
>>     check_freshness         1
>>     contact_groups          admins) to ensure that they contain valid
>>     notification_interval   2tions.  If you are upgrading from a previous
>>     notification_period     24x7 be aware that some variables/definitions
>>     notifications_enabled   1odified in this version.  Make sure to read
>>     register                1arding the config files, as well as the
>> } 'Whats New' section to find out what has changed.
>> "@
>> $config | Out-File -FilePath "MyWindowsHost001.cfg" -Encoding ASCII
PS C:\Users\tanma> docker cp MyWindowsHost001.cfg nagios4:/opt/nagios/etc/servers/MyWindowsHost001.cfg
Successfully copied 2.56kB to nagios4:/opt/nagios/etc/servers/MyWindowsHost001.cfg
PS C:\Users\tanma> docker exec -it nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg

Nagios Core 4.5.12
Copyright (c) 2009-present Nagios Core Development Team and Community Contributors
Copyright (c) 1999-2009 Ethan Galstad
Last Modified: 2026-03-25
License: GPL

Website: https://www.nagios.org
Reading configuration data...
   Read main config file okay...
   Read object config files okay...

Running pre-flight check on configuration data...
```

```
Administrator: Windows PowerShell
>> $config | Out-File -FilePath "MyWindowsHost001.cfg" -Encoding ASCII
PS C:\Users\tanma> docker cp MyWindowsHost001.cfg nagios4:/opt/nagios/etc/servers/MyWindowsHost001.cfg
Successfully copied 2.56kB to nagios4:/opt/nagios/etc/servers/MyWindowsHost001.cfg
PS C:\Users\tanma> docker exec -it nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg

Nagios Core 4.5.12
Copyright (c) 2009-present Nagios Core Development Team and Community Contributors
Copyright (c) 1999-2009 Ethan Galstad
Last Modified: 2026-03-25
License: GPL

Website: https://www.nagios.org
Reading configuration data...
   Read main config file okay...
   Read object config files okay...

Running pre-flight check on configuration data...

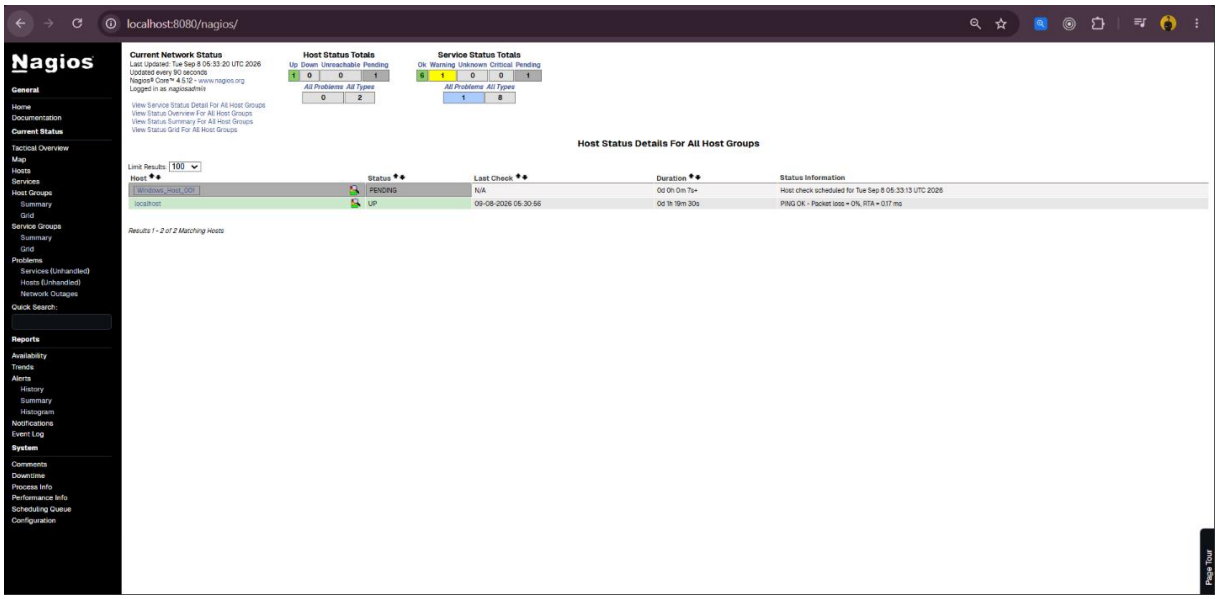
Checking objects...
        Checked 8 services.
        Checked 2 hosts.
        Checked 2 host groups.
        Checked 0 service groups.
        Checked 1 contacts.
        Checked 1 contact groups.
        Checked 23 commands.
        Checked 5 time periods.
        Checked 0 host escalations.
        Checked 0 service escalations.
Checking for circular paths...
        Checked 2 hosts
        Checked 0 service dependencies
        Checked 0 host dependencies
        Checked 5 timeperiods
Checking global event handlers...
Checking obsessive compulsive processor commands...
Checking misc settings...

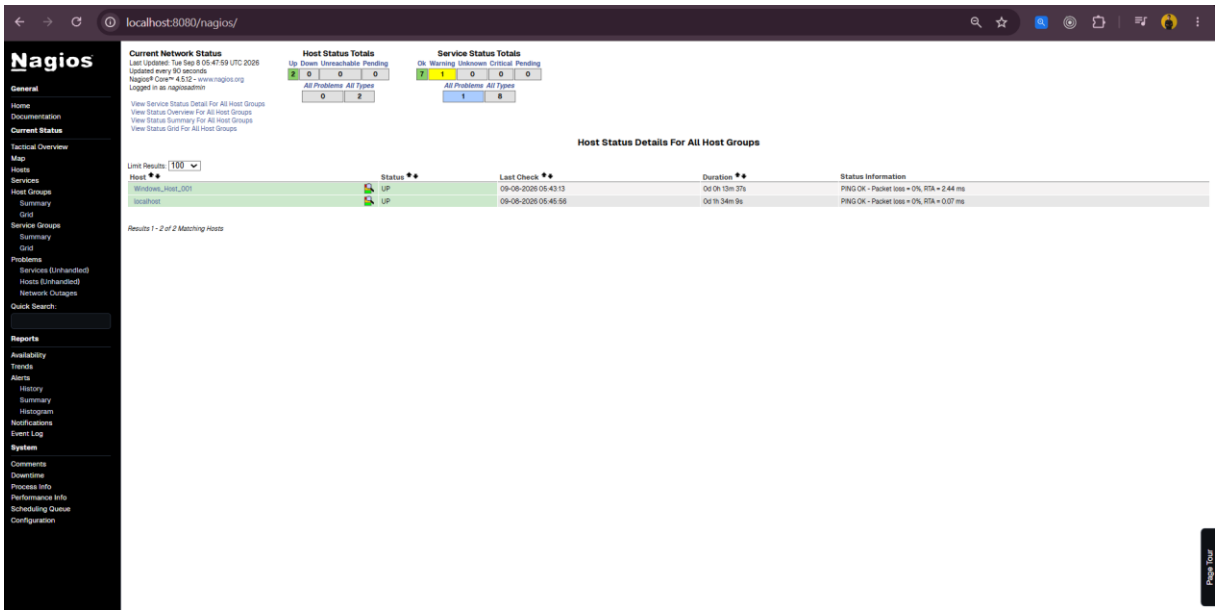
Total Warnings: 0
Total Errors:   0

Things look okay - No serious problems were detected during the pre-flight check
PS C:\Users\tanma> docker restart nagios4
nagios4
```

9 - Check Host in Nagios Web Interface

Open the Nagios web interface (http://localhost:8080/nagios/) and check that the new Windows host has been added and is being monitored under Current Status -> Hosts.





Conclusion :

This experiment successfully demonstrates the implementation of continuous IT infrastructure monitoring using Nagios within a containerized Docker environment. By deploying the Nagios Core image and configuring the NSClient++ remote agent on a Windows host, a functional active monitoring architecture is established. This setup enables the ongoing tracking of network issues and server performance, allowing teams to identify root causes and troubleshoot problems before they impact critical business processes. I am fully prepared to assist you with any upcoming DevOps pipelines, ML lifecycle tools, or additional engineering assignments you need to tackle next.